

OOP期末复习资料（按 13 讲整理）

来源：当前目录 13 份 OOP 课程 PPTX。整理目标是期末复习：按讲次梳理概念、语法规则、代码模式、易错点和设计模式思想。

总复习主线

本课程从“如何写出更好的软件”展开，核心标准是简单性、清晰性、普遍性。C++ 的 OOP 学习并不是只把代码塞进类里，而是学习如何抽象对象、隐藏实现、建立关系、复用代码、管理资源，并在需求变化时控制修改范围。

课程核心线索如下：

1. 从过程式程序到对象：类型不仅是存储和表示，还包含可执行的操作。
2. 封装：把数据和服务放在一起，用 public 接口暴露能力，用 private/protected 隐藏实现。
3. 生命周期：构造、析构、拷贝、移动、赋值共同决定对象资源是否安全。
4. 关系建模：组合表达 has-a，继承表达 is-a 或接口实现。
5. 动态绑定：用继承、虚函数、基类指针/引用实现运行期多态。
6. 泛型：用模板和 STL 在编译期复用源代码。
7. 设计模式：用稳定结构吸收变化，做到对扩展开放、对修改封闭。

第 1 讲 绪论与编程环境初步

1.1 OOP 的定位

面向对象程序设计是一种程序设计方法论，重点不是“怎么计算”，而是“怎么看问题”。FOP 更强调通过变量、判断、循环、函数描述计算过程；OOP 更强调通过对象、属性、服务、协作来描述问题世界。

对象是对现实世界中实际事物或概念的抽象。对象具有：

- 静态特征：可用数据描述的属性。
- 动态特征：对象能表现的行为或服务。
- 独立性：对象是构成系统的相对独立单位。

抽象是从大量事物中舍弃非本质特征，提取共同、本质特征的过程。课程中强调两类抽象：

- 过程抽象：把一个确定功能的操作序列视为单一实体，例如函数。
- 数据抽象：根据数据支持的操作定义数据类型，并限制数据只能通过这些操作被观察和修改。

1.2 类型观念

程序语言中的类型类似数学中的数据概念：类型不仅有值域，还规定了允许的操作。例如整数除法 $3 / 4 == 0$ ，浮点除法 $3.2 / 4.0 == 0.8$ 。因此可把类型理解为：

类型 = 数据的存储与表示 + 数据支持的操作

OOP 的关键是允许程序员定义新类型，使其像内置类型一样有明确的数据、操作、生命周期和接口。

1.3 为什么仍要学习 C++ 和 OOP

C++ 的优势包括跨平台、运行效率高、接近底层、适合系统软件和大型工程底层库。它的代价是复杂，需要程序员理解内存、生命周期、对象模型、编译链接等底层机制。

AI 时代仍要学 OOP 的原因：

- AI 可以辅助生成局部代码，但项目的抽象边界、接口设计、模块依赖和可扩展架构仍需人判断。
- OOP 能训练架构思维，尤其是如何让代码在变化中保持清晰和可维护。
- 期末和作业中会涉及代码审查、正确性、可读性、可扩展性评价。

1.4 课程内容台阶

课程按如下层次推进：

1. C/C++ 面向过程基础。
2. 类与对象。
3. 对象创建、销毁、引用、复制。
4. 组合与继承。
5. 虚函数与多态。
6. 模板与泛型编程。
7. STL、函数对象、智能指针。
8. 设计模式：行为型和结构型模式。

1.5 命令行、编译器、IDE 基础

命令行是通过文本命令控制操作系统的接口。常见命令：

功能	Windows	Linux/macOS
显示当前目录	cd	pwd
查看文件	dir	ls
进入目录	cd path	cd path
新建目录	mkdir name	mkdir name
新建文件	type nul>a.cpp	touch a.cpp
删除文件	del a.cpp	rm a.cpp
复制文件	copy a.cpp dst\	cp a.cpp dst/
移动文件	move a.cpp dst\	mv a.cpp dst/

编译命令基本形式：

```
g++ test.cpp -o test
g++ test.cpp -o test -O2
```

环境变量 `PATH` 告诉系统去哪些目录查找可执行程序。若命令行提示找不到 `g++`，通常需要安装编译器并把其 `bin` 目录加入 `PATH`。

VS Code 常见配置文件：

- `c_cpp_properties.json`：头文件目录、C++ 标准等。
- `tasks.json`：编译命令、编译参数。
- `launch.json`：调试目标、运行参数、断点配置。

第 2 讲 编译、链接、多文件工程、宏与 Make

2.1 命令行参数

`main` 可以接收命令行参数：

```
int main(int argc, char** argv) {  
    // argc: 参数个数，含程序名  
    // argv[0]: 程序名  
    // argv[1], argv[2], ...: 命令行参数字符串  
}
```

例如运行：

```
./ex2 4 5
```

则 `argc == 3`，`argv[0]` 是程序名，`argv[1] == "4"`，`argv[2] == "5"`。使用前必须检查边界：

```
int main(int argc, char** argv) {  
    if (argc != 3) {  
        std::cout << "Usage: " << argv[0] << " op1 op2\n";  
        return 1;  
    }  
    int a = std::atoi(argv[1]);  
    int b = std::atoi(argv[2]);  
    std::cout << a + b << std::endl;  
}
```

复习要点：永远考虑边界和异常情况，不能直接假设 `argv[1]` 一定存在。

2.2 编译和链接

源程序到可执行程序的基本过程：

源文件 .cpp -> 编译器 -> 目标文件 .o/.obj -> 链接器 -> 可执行文件

常见命令：

```
g++ -c ex1.cpp -o ex1.o      # 只编译，不链接
g++ ex1.o -o ex1             # 链接
g++ main.cpp func.cpp -o ex  # 直接编译并链接多个文件
```

编译阶段检查语法、类型、声明是否可见；链接阶段查找函数和全局变量的定义，完成符号解析和地址重定位。常见链接错误：

- **undefined reference**：声明存在，但链接时找不到定义。
- **multiple definition**：同一个函数或全局变量被定义多次。

2.3 声明与定义

函数可以多次声明，但只能定义一次：

```
int ADD(int a, int b);           // 声明
int ADD(int a, int b) { return a + b; } // 定义
```

变量也有声明和定义：

```
int x;           // 定义，会分配存储
extern int x;     // 声明，不分配存储
int arr[100];    // 定义数组
extern int arr[100]; // 声明数组
```

全局变量跨文件共享时，通常在头文件写 **extern** 声明，在一个 **.cpp** 中给出唯一定义：

```
// num.h
extern int a;

// num.cpp
#include "num.h"
int a = 1;
```

2.4 头文件

头文件用于统一声明，避免在多个源文件中重复手写函数、类、全局变量声明。**#include** 的本质是预处理时文本替换，把被包含文件内容复制到当前位置。

头文件中通常放：

- 函数声明。
- 类定义。
- 模板定义。
- `extern` 变量声明。
- 常量、内联函数等可安全多次包含的内容。

头文件中通常不要放普通函数定义、普通全局变量定义，否则多个 `.cpp` 包含后会导致重复定义。

防止重复包含：

```
#ifndef FUNC_H
#define FUNC_H
int ADD(int a, int b);
#endif
```

或：

```
#pragma once
int ADD(int a, int b);
```

`#pragma once` 更简洁，现代编译器普遍支持；`header guard` 更标准、可移植。

2.5 宏定义

宏是预处理阶段的文本替换：

```
#define PI 3.1415926535
#define sqr(x) ((x) * (x))
```

宏没有类型检查，容易出现优先级和副作用问题。例如：

```
#define M(y) y*y+3*y
M(1+1) // 展开为 1+1*1+1+3*1+1，结果不是按函数调用理解
```

C++ 中优先用：

- `const` / `constexpr` 替代常量宏。
- `inline` 函数替代函数宏。
- `#ifdef DEBUG` 等条件编译场景仍可使用宏。

2.6 C++11 编译

课程默认使用 C++11 标准。常用编译参数：

```
g++ -std=c++11 ex.cpp -o ex
```

`-g` 生成调试信息，`-O2` 开启优化。优化后程序行为变化常常暴露未定义行为，例如数组越界、未初始化变量等。

2.7 Makefile

Make 用依赖关系自动化编译。规则格式：

```
target: prerequisites
    commands
```

注意命令前必须是 Tab。例：

```
all: test

test: product.o sum.o main.o functions.h
    g++ product.o sum.o main.o -o test

product.o: product.cpp functions.h
    g++ -c product.cpp -o product.o

sum.o: sum.cpp functions.h
    g++ -c sum.cpp -o sum.o

main.o: main.cpp functions.h
    g++ -c main.cpp -o main.o

clean:
    rm *.o test
```

自动变量：

- `$@`：当前目标。
- `$$`：所有依赖。
- `$<`：第一个依赖。

模式规则：

```
%.o: %.cpp
    g++ -c $< -o $@
```

`.PHONY` 用于声明伪目标，避免与同名文件冲突：

```
.PHONY: all clean
```

2.8 GDB 调试

用 `-g` 编译：

```
g++ -g a.cpp -o a.out
gdb a.out
```

常用命令：

命令	含义
<code>run</code>	运行
<code>break 10</code>	第 10 行设断点
<code>break 10 if k==2</code>	条件断点
<code>delete break 1</code>	删除断点
<code>watch x</code>	监视变量变化
<code>continue</code>	继续运行
<code>step</code>	单步进入
<code>next</code>	单步跳过
<code>print x</code>	打印变量
<code>display x</code>	持续显示
<code>list</code>	显示源码
<code>quit</code>	退出

第 3 讲 封装与接口

3.1 函数重载

函数重载指同一作用域中多个同名函数具有不同参数列表：

```
void print(const char* msg);
void print(std::string msg);
void print(int score);
void print(float score);
```

编译器根据实参类型选择调用版本。重载只看函数名和参数列表，不能只靠返回值区分：

```
int f(int);  
float f(int); // 错误：不能仅返回值不同
```

实参与形参不完全匹配时，编译器可能进行内置类型转换。这会带来二义性或意外匹配，复习时要注意函数调用到底匹配哪个版本。

3.2 默认参数

函数参数可设置缺省值：

```
void print(const char* msg = "hello");  
void print(char* name, int score, char* msg = "pass");
```

规则：

- 有默认值的参数必须从右向左连续出现。
- 默认参数可能与函数重载产生二义性。
- 缺省值通常写在声明处，不要在多个声明中重复给出不同默认值。

3.3 auto 和 decltype

auto 由编译器根据初始化表达式推导类型：

```
auto i = 3;          // int  
auto f = 4.0f;       // float  
auto it = vec.begin();
```

限制：

- auto 变量定义时必须初始化。
- 同一条声明中的多个 auto 变量必须推导为同一类型。
- C++11 中函数返回类型可用尾置返回：

```
auto func(int x, int y) -> decltype(x + y);
```

decltype(expr) 获取变量或表达式类型，常用于模板和尾置返回类型：

```
template <typename Tx, typename Ty>  
auto multiply(Tx x, Ty y) -> decltype(x * y) {  
    return x * y;  
}
```


3.4 动态内存与空指针

动态申请和释放：

```
int* p = new int(10);
delete p;

int* arr = new int[10];
delete[] arr;
```

`new` 与 `delete`、`new[]` 与 `delete[]` 必须配套。错误配对会导致析构次数错误、内存泄漏或未定义行为。

空指针：

- C++ 中 `NULL` 常被定义为 `0`，可能参与整数重载而非指针重载。
- C++11 引入 `nullptr`，表示严格空指针，优先使用。

3.5 范围 for

C++11 的范围 for：

```
for (auto x : container) { } // x 是元素副本
for (auto& x : container) { } // x 是元素引用，可修改
for (const auto& x : container) { } // 只读且避免拷贝
```

3.6 类与对象

类是用户自定义类型，是包含数据和函数的特殊结构：

```
class Agent {
private:
    std::string memory[6];
public:
    void plan(std::string goal);
};
```

类中函数称为成员函数，数据称为成员变量。对象是类类型的变量。

访问成员：

```
Agent a;
a.plan("goal");

Agent* p = &a;
p->plan("goal");
```

在类外只能访问 `public` 成员。

3.7 接口与实现分离

复杂成员函数一般声明在头文件、定义在 `.cpp`：

```
// agent.h
class Agent {
public:
    void plan(std::string goal);
};

// agent.cpp
#include "agent.h"
void Agent::plan(std::string goal) {
    // ...
}
```

类外定义成员函数需要 `ClassName::` 限定。

3.8 访问控制

- `public`：类外可访问，是对象对外接口。
- `private`：仅类内成员和友元可访问，隐藏实现。
- `protected`：类内和派生类可访问，类外不可访问。

封装的核心不是“数据都写 `private`”这么简单，而是只暴露必要接口，让对象状态只通过受控操作改变。

3.9 `this` 指针

每个非静态成员函数都有隐含参数 `this`，指向当前对象：

```
class A {
    int data;
public:
    void set(int data) {
        this->data = data;
    }
};
```

`this` 常用于：

- 区分形参和成员变量。
- 返回当前对象引用：`return *this;`
- 在成员函数中把当前对象传给其他函数。

3.10 内联函数

`inline` 建议编译器把函数体展开到调用处，减少函数调用开销：

```
inline int max(int a, int b) {  
    return a > b ? a : b;  
}
```

内联优点：

- 有类型检查，比宏安全。
- Debug 下可像普通函数一样调试。

注意：

- `inline` 是建议，不是命令。
- 不建议内联大函数、递归函数、包含复杂循环或复杂控制结构的函数。
- 类内定义的短成员函数通常隐式内联。

第 4 讲 创建与销毁、引用、运算符重载

4.1 构造函数

构造函数负责对象初始化，由编译器在创建对象处自动调用。

规则：

- 无返回值类型。
- 函数名与类名相同。
- 可以重载。
- 应使用初始化列表初始化成员。

```
class Student {  
    int ID;  
public:  
    Student(int id) : ID(id) {}  
    Student(int year, int order) : ID(year * 10000 + order) {}  
};
```

初始化列表：

```
ClassName(args) : member1(value1), member2(value2) {}
```

成员初始化顺序由成员在类中声明的顺序决定，不由初始化列表书写顺序决定。这是常考易错点：

```
class Student {  
    int ID1;  
    int ID2;
```

```
public:
    Student(int id) : ID2(id), ID1(ID2) {} // 先初始化 ID1, 此时 ID2 未初始化
};
```

4.2 委派构造与就地初始化

C++11 支持委派构造:

```
class Info {
public:
    Info() { Init(); }
    Info(int i) : Info() { id = i; }
private:
    int id;
    void Init() {}
};
```

C++11 支持成员就地初始化:

```
class A {
    int a = 1;
    double b{2.0};
};
```

4.3 默认构造函数

默认构造函数是不带参数, 或所有参数都有默认值的构造函数:

```
class A {
public:
    A() {}
};
```

创建对象:

```
A a;           // 调用默认构造函数
A b = A();     // 调用默认构造函数
A c();         // 注意: 这是函数声明, 不是对象定义
```

若用户没有定义任何构造函数, 编译器可能隐式合成默认构造函数; 一旦用户定义了其他构造函数, 编译器不会再自动合成默认构造函数。可显式要求或删除:

```
Student() = default;  
Student(char) = delete;
```

= delete 可阻止不希望发生的隐式类型转换。

4.4 对象数组初始化

无参定义对象数组要求类有默认构造函数：

```
A a[50];  
A b[3] = {1, 3, 5};  
A c[3] = {A(1, 2), A(3, 5), A(0, 7)};
```

4.5 析构函数

析构函数负责对象生命周期结束时的清理：

```
class Classroom {  
    int* ID_list;  
public:  
    Classroom() : ID_list(nullptr) {}  
    ~Classroom() { delete[] ID_list; }  
};
```

规则：

- 名称为 ~ClassName。
- 无返回值，无参数。
- 一个类只能有一个析构函数。
- 编译器在对象生命周期结束时自动调用。
- 隐式析构函数不会自动 delete 指针成员。

析构顺序：先执行本对象析构函数体，再析构成员对象；构造顺序相反，先构造成员对象，再执行本对象构造函数体。

4.6 局部对象、全局对象、静态对象生命周期

局部对象：

- 执行到定义处构造。
- 离开作用域析构。

全局对象：

- main() 前构造。
- main() 返回后析构。

- 同一编译单元按定义顺序初始化，不同编译单元间顺序不确定。

建议尽量少用有依赖关系的全局对象，否则会遇到静态初始化顺序问题。

4.7 引用

引用是具名变量的别名：

```
int v0;  
int& v1 = v0;
```

规则：

- 引用必须定义时初始化。
- 引用初始化后不能改绑到另一个对象。
- 不存在空引用。
- 引用作为参数可避免拷贝并允许修改实参。

引用参数：

```
void swap(int& a, int& b) {  
    int tmp = a;  
    a = b;  
    b = tmp;  
}
```

引用返回：

```
int a[3] = {1, 3, 5};  
int& get(int i) { return a[i]; }
```

不能返回局部临时变量的引用。

4.8 运算符重载

运算符重载让自定义类型以接近内置类型的方式使用。形式：

```
A operator+(A a, A b);           // 全局函数  
class A {  
public:  
    A operator+(const A& rhs) const; // 成员函数  
};
```

常见建议：

- `+=` 通常返回 `A&`，便于链式调用。
- `+` 通常返回新对象。
- 前缀 `++` 返回修改后的对象引用。
- 后缀 `++` 用虚参数 `int` 区分，返回修改前副本。

```
class Test {
public:
    int data = 1;
    Test& operator++() {
        ++data;
        return *this;
    }
    Test operator++(int) {
        Test old = *this;
        ++data;
        return old;
    }
};
```

只能作为成员函数重载的运算符：

=? 重点：=, [], (), -> 必须是成员函数

课程中强调 `=`, `[]`, `()`, `->` 只能通过成员函数重载。

4.9 函数调用和下标运算符

函数调用运算符：

```
class Add {
public:
    int operator()(int a, int b) { return a + b; }
};
Add add;
add(1, 2);
```

下标运算符若要出现在等号左边，返回引用：

```
int& operator[](int index);
```

4.10 流运算符重载

输入输出流运算符通常写成全局函数，返回流引用以支持链式调用：

```
std::istream& operator>>(std::istream& in, Test& dst);  
std::ostream& operator<<(std::ostream& out, const Test& src);
```

若需要访问私有成员，可声明为友元。

第 5 讲 友元、静态成员、常量成员、动态对象

5.1 友元

友元函数或友元类可以访问类的 `private` 和 `protected` 成员：

```
class A {  
    int data;  
    friend void foo(A& a);  
};
```

要点：

- 友元声明只能写在类内。
- 友元函数不是当前类的成员函数。
- 当前类自己的成员函数不需要声明为友元。
- 友元声明与其写在 `private` 还是 `public` 区域无关。
- 普通函数可以同时是多个类的友元。
- 友元类表示该类所有成员函数都是友元。

友元关系特性：

- 不对称：A 声明 B 是友元，不代表 A 能访问 B。
- 不传递：朋友的朋友不是朋友。
- 不继承：友元关系不因继承自动传给派生类。

5.2 C/C++ 中的 `static`

C 风格静态变量/函数：

- 静态局部变量：生命周期持续到程序结束，只初始化一次，作用域仍在函数内。
- 静态全局变量/静态函数：内部链接，只能在当前文件使用，避免跨文件命名冲突。

5.3 静态数据成员

静态数据成员属于类，被所有对象共享：

```
class Test {  
public:  
    static int count;  
    Test() { ++count; }
```



```
};

int Test::count = 0; // 类外定义
```

访问方式：

```
Test::count;
obj.count; // 也可，但不推荐
```

静态数据成员应在头文件声明，在 `.cpp` 中定义。若在头文件普通定义，被多个源文件包含后会导致重复定义。

5.4 静态成员函数

```
class Test {
public:
    static int count;
    static int getCount() { return count; }
};
```

静态成员函数属于类，不属于某个对象，因此：

- 没有 `this` 指针。
- 不能直接访问非静态成员。
- 可访问静态成员。

5.5 常量数据成员和常量成员函数

常量数据成员生命周期内不可修改，必须在初始化列表或就地初始化中初始化：

```
class Student {
    const int ID;
public:
    Student(int id) : ID(id) {}
};
```

常量成员函数：

```
class Student {
    int ID;
public:
    int MyID() const { return ID; }
};
```

`ReturnType func() const` 表示函数不修改对象状态。常量对象只能调用常量成员函数。

注意区别：

```
const int f();      // 返回值是 const int
int f() const;     // 成员函数不修改对象
```

5.6 常量静态成员

静态常量成员兼具类共享和不可修改属性。一般仍需类外定义，但 `static const int` 和 `enum` 等整数常量可在类内就地初始化：

```
class Foo {
    static const int i = 3;
    static const char* cs;
};
const char* Foo::cs = "foo";
```

不存在“常量静态函数”这一概念。

5.7 参数对象构造析构

对象按值传参会构造形参对象，函数结束时析构形参：

```
void fun(A b); // 调用时拷贝构造 b，结束时析构 b
```

对象引用传参不会构造新对象，也不会额外析构：

```
void fun(A& b); // b 是实参别名
```

若类含指针成员，默认拷贝可能导致两个对象指向同一块内存，最终重复释放。此处为后续拷贝构造、赋值运算符、移动语义埋下伏笔。

5.8 new 和 delete

单对象：

```
A* p = new A(args);
delete p;
```

对象数组：

```
A* p = new A[3];
delete[] p;
```

`new[]` 会为每个元素调用构造函数，`delete[]` 会为每个元素调用析构函数。`new[]` 配 `delete` 只析构一次或行为未定义，是严重错误。

第 6 讲 常量引用、拷贝、移动、赋值、类型转换

6.1 常量引用与最小特权原则

若函数只读取参数，应使用常量引用：

```
void add(const int& a, const int& b);
void print(const BigObject& obj);
```

好处：

- 避免对象拷贝。
- 不允许函数修改实参。
- 可绑定右值，提升调用灵活性。

6.2 拷贝构造函数

拷贝构造函数形式：

```
class Person {
public:
    Person(const Person& src);
};
```

常见调用时机：

1. 用一个对象初始化新对象：`Test b(a);`、`Test c = a;`
2. 函数按值传递对象：`void f(Test a);`
3. 函数按值返回对象：`return tmp;`，但可能被返回值优化消除。

若未显式定义，编译器会合成拷贝构造函数：对类成员递归调用拷贝构造，对基础类型执行值拷贝。若类有裸指针成员，默认拷贝常导致浅拷贝问题。

典型危险：

```
class Pointer {
    int* m_arr;
public:
```

```

    Pointer(int n) : m_arr(new int[n]) {}
    ~Pointer() { delete[] m_arr; }
};

Pointer a(5);
Pointer b = a; // 默认浅拷贝, a 和 b 指向同一数组

```

解决方式:

- 自定义深拷贝。
- 禁用拷贝: `Pointer(const Pointer&) = delete;`
- 使用智能指针或 STL 容器。

6.3 左值、右值、右值引用

左值: 有名字、可取地址的表达式。右值: 临时值、字面量、表达式结果、函数返回值等通常不可取地址。

```

int a = 1;        // a 是左值, 1 是右值
int b = a + 1;    // a+1 是右值

```

引用绑定规则:

- `T&` 绑定非常量左值。
- `const T&` 可绑定左值、常量左值、右值。
- `T&&` 绑定右值。
- 右值引用变量本身是左值。

```

int&& r = a + b; // 可以
// int&& r2 = a; // 不可以
const int& cr = a + b; // 可以

```

6.4 移动构造函数

移动构造函数使用右值引用参数:

```

class Test {
    int* buf;
public:
    Test(Test&& other) : buf(other.buf) {
        other.buf = nullptr;
    }
};

```

移动语义不是复制资源, 而是“接管”即将销毁对象的资源。必须把源对象置于可析构状态, 例如把指针设为 `nullptr`, 避免重复释放。

`std::move` 不移动对象，只把左值转换为右值引用，从而允许调用移动构造或移动赋值：

```
Test b = std::move(a);
```

`std::move(a)` 后，`a` 仍存在，但资源可能被转移，只应赋新值或析构，不应再依赖原值。

6.5 拷贝赋值与移动赋值

拷贝赋值用于已存在对象之间赋值：

```
ClassName& operator=(const ClassName& right) {  
    if (this != &right) {  
        // 释放旧资源，深拷贝新资源  
    }  
    return *this;  
}
```

移动赋值：

```
ClassName& operator=(ClassName&& right) {  
    if (this != &right) {  
        // 释放旧资源  
        // 接管 right 资源  
        right.ptr = nullptr;  
    }  
    return *this;  
}
```

必须处理自赋值，返回 `*this` 支持链式赋值。

编译器可能自动合成：

- 默认构造函数。
- 拷贝构造函数。
- 移动构造函数。
- 拷贝赋值运算符。
- 移动赋值运算符。
- 析构函数。

掌握“Rule of Three/Five”：如果类需要自定义析构、拷贝构造或拷贝赋值，通常也需要认真处理另外几个特殊成员。

6.6 自动类型转换

自动类型转换可通过转换构造函数或类型转换运算符实现：

```
class Dst {
public:
    Dst(const Src& s); // Src -> Dst
};

class Src {
public:
    operator Dst() const; // Src -> Dst
};
```

类型转换运算符规则：

```
operator int() const; // 无返回类型声明，无参数
```

错误写法：

```
int operator int() const;    // 错
operator int(int = 0) const; // 错
```

6.7 explicit

explicit 禁止隐式类型转换，只允许显式转换：

```
class Dst {
public:
    explicit Dst(const Src& s);
};

Dst d1(s);           // 可以
Dst d2 = s;          // 不可以
Dst d3 = static_cast<Dst>(s); // 可以
```

复习判断：如果隐式转换可能让代码读者误解，优先使用 **explicit**。

6.8 强制类型转换

C++ 四种命名转换：

转换	用途
<code>const_cast</code>	去除或添加 <code>const/volatile</code> 属性
<code>static_cast</code>	静态类型转换，编译期检查转换路径
<code>dynamic_cast</code>	多态类层次中的运行期安全转换

转换

用途

`reinterpret_cast` 重新解释二进制表示，风险最高

第 7 讲 组合与继承

7.1 类之间的两种基本关系

组合：has-a，整体-部分关系。例如汽车有轮子、引擎。

继承：is-a，一般-特殊关系。例如矩形是一种形状。

建模时首先判断关系：

- 如果 A 是 B 的组成部分，用组合。
- 如果 A 是一种 B，用 public 继承。
- 如果只是复用实现，优先组合，谨慎 private 继承。

7.2 组合

组合用成员对象表达：

```
class Wheel {};  
class Engine {};  
  
class Car {  
private:  
    Wheel w;  
    Engine e;  
};
```

构造顺序：

1. 先构造成员对象。
2. 再执行当前对象构造函数体。
3. 成员构造顺序由声明顺序决定。
4. 析构顺序与构造顺序相反。

成员对象若需要参数，应在初始化列表中构造：

```
class Car {  
    Wheel w;  
public:  
    Car(int n) : w(n) {}  
};
```

隐式拷贝构造和赋值会递归调用成员对象的拷贝构造和赋值；基础类型按值复制。

7.3 继承基础

```
class Base {};  
class Derived : public Base {};
```

术语：

- 基类 / 父类：被继承的类。
- 派生类 / 子类：继承得到的新类。

继承方式：

- `public`：表达 is-a，最常用。
- `private`：表达用基类实现派生类，不保留 is-a 接口关系；通常可用组合替代。
- `protected`：少用。

`class Derived : Base` 默认 `private` 继承；`struct` 默认 `public` 继承。

7.4 不会普通继承的成员

通常认为以下成员不会作为普通成员被继承：

- 构造函数：派生类构造函数负责调用基类构造函数。C++11 可 `using Base::Base` 继承构造函数。
- 析构函数：析构时先派生类，再基类。
- 赋值运算符：不会继承参数为基类的赋值运算符，但可自动合成派生类赋值运算符。
- 友元函数：不是成员，不继承。

7.5 派生类构造和析构

构造派生类对象时，先构造基类部分，再构造派生类部分：

```
class Derived : public Base {  
public:  
    Derived(int x) : Base(x) {}  
};
```

若初始化列表未显式调用基类构造函数，编译器尝试调用基类默认构造函数。若基类没有可访问默认构造函数，编译失败。

析构顺序相反：先派生类析构函数体，再基类析构函数。

7.6 继承构造函数

C++11 可用：

```
class Derived : public Base {  
public:
```



```
using Base::Base;

};
```

注意：

- 若基类某构造函数私有，不能继承使用。
- 使用继承构造函数后，编译器不一定再合成派生类默认构造函数。

7.7 成员访问权限与继承方式

基类 `private` 成员：派生类成员函数也不能直接访问。

基类 `protected` 成员：派生类成员函数可访问，类外不可访问。

基类 `public` 成员：是否在派生类对象上公开取决于继承方式。

简表：

基类成员	public 继承后	protected 继承后	private 继承后
public	public	protected	private
protected	protected	protected	private
private	不可直接访问	不可直接访问	不可直接访问

`private` 继承后如需重新公开某个基类成员，可在 `public` 区域使用：

```
using Base::baseFunc;
```

7.8 重载、重写隐藏

重载 `overload`：

- 同一作用域。
- 函数名相同。
- 参数列表不同。
- 静态多态。

重写隐藏 `redefining/hiding`：

- 派生类定义了与基类同名函数。
- 会隐藏基类所有同名重载版本，即使参数不同。

```
class Base {
public:
    void f();
    void f(int);
};

class Derived : public Base {
```

```
public:
    void f(double); // 隐藏 Base::f 和 Base::f(int)
};
```

恢复基类重载：

```
class Derived : public Base {
public:
    using Base::f;
    void f(double);
};
```

7.9 多重继承

```
class IOFile : public InputFile, public OutputFile {};
```

优点：可结合多个接口，符合某些直觉模型。

问题：

- 二义性：多个基类有同名成员。
- 数据冗余：菱形继承中同一基类子对象可能存在多份。

实践建议：最多继承一个非抽象类，可以继承多个抽象接口。

第 8 讲 虚函数

8.1 向上类型转换

派生类对象、指针或引用转换为基类对象、指针或引用叫向上类型转换。只对 public 继承表达的 is-a 关系自然成立。

```
Derived d;
Base b = d;           // 对象转换，会切片
Base* p = &d;         // 指针转换，不切片
Base& r = d;          // 引用转换，不切片
```

8.2 对象切片

当派生类对象被转换为基类对象时，只保留基类子对象，派生类新增部分被切掉：

```
Base b = derived; // slicing
```

对象切片会丢失派生类状态，也不能实现多态。若想保留实际对象类型，应使用指针或引用。

8.3 早绑定与晚绑定

绑定是把函数调用与函数体联系起来。

- 早绑定：编译期确定调用哪个函数。普通成员函数默认如此。
- 晚绑定：运行期根据对象真实类型确定调用哪个函数。C++ 通过虚函数实现。

8.4 虚函数

基类中声明 `virtual`：

```
class Instrument {
public:
    virtual void play() { std::cout << "Instrument\n"; }
};

class Wind : public Instrument {
public:
    void play() override { std::cout << "Wind\n"; }
};
```

通过基类指针或引用调用虚函数时，根据对象真实类型执行：

```
void tune(Instrument& i) {
    i.play(); // 若 i 实际为 Wind，调用 Wind::play
}
```

虚函数实现通常依赖虚函数表和虚表指针。对象中保存虚表指针，运行期通过虚表找到实际函数。

8.5 多态成立条件

运行期多态需要三个条件：

1. 继承。
2. 虚函数。
3. 通过基类指针或引用调用。

若用对象直接调用，即使函数是虚函数，也按对象静态类型决定。

8.6 构造函数、析构函数与虚函数

构造函数不能是虚函数。原因是构造过程中对象类型尚未完整形成，虚表也在随构造阶段变化。

析构函数可以且经常应该是虚函数。若基类指针指向派生类对象并通过 `delete` 删除，基类析构函数必须是虚的，否则只调用基类析构，派生类资源可能泄露：

```
class Base {  
public:  
    virtual ~Base() {}  
};
```

在构造函数和析构函数中调用虚函数，不会表现为完整派生类多态，通常按当前构造/析构阶段的类来绑定，设计上应避免依赖这种行为。

8.7 重写覆盖、`override` 与 `final`

重写覆盖 `overriding`：派生类重新定义基类虚函数，且函数签名兼容。

推荐使用 `override`，让编译器检查是否真正重写：

```
class Derived : public Base {  
public:  
    void f() override;  
};
```

`final` 禁止继续重写或继承：

```
class Derived final : public Base {};  
void f() final;
```

第 9 讲 抽象类、向下转换、多态、模板

9.1 纯虚函数与抽象类

纯虚函数：

```
virtual ReturnType func(Args) = 0;
```

含至少一个纯虚函数的类是抽象类。抽象类不能定义对象，主要用于规定接口：

```
class Shape {  
public:  
    virtual double area() const = 0;  
};
```

派生类若未实现所有纯虚函数，仍是抽象类。

纯虚函数也可以在类外定义默认函数体，派生类可显式调用：

```
void Base::f() { /* default */ }
```

9.2 纯虚析构函数

析构函数也可声明为纯虚：

```
class Base {  
public:  
    virtual ~Base() = 0;  
};  
Base::~Base() {}
```

纯虚析构函数必须有函数体，因为派生类析构后一定会调用基类析构。

把析构函数设为纯虚可使基类成为抽象类。

9.3 向下类型转换

基类指针/引用转派生类指针/引用叫向下类型转换。向下转换有风险，因为基类指针不一定真的指向目标派生类对象。

安全方式：

```
Derived* p = dynamic_cast<Derived*>(basePtr);  
if (p) { p->derivedOnly(); }
```

`dynamic_cast`：

- 要求类有虚函数。
- 指针转换失败返回 `nullptr`。
- 引用转换失败抛出 `std::bad_cast`。
- 运行期检查，更安全但有开销。

`static_cast`：

```
Derived* p = static_cast<Derived*>(basePtr);
```

只在编译期检查继承关系，不验证对象真实类型。除非非常确定，否则向下转换优先使用 `dynamic_cast`。

原则：

1. 向上转换通常安全。
2. 向下转换优先 `dynamic_cast`。

3. 避免对象之间的上下转换，防止切片。

9.4 多重继承与接口

多重继承适合继承多个抽象接口：

```
class CanSpeak { public: virtual void speak() = 0; };
class CanMove { public: virtual void move() = 0; };
class Dog : public CanSpeak, public CanMove {};
```

最佳实践：一个实现基类 + 多个接口基类，降低数据冗余和二义性风险。

9.5 多态

多态指同一基类接口调用，在运行时因对象真实派生类型不同而表现不同。

优点：

- 上层代码只依赖抽象接口。
- 新增派生类时，调用方代码可不变。
- 提升复用性、扩展性和维护性。

示例：

```
class Animal {
public:
    virtual void speak() = 0;
    virtual void motion() = 0;
};

void action(Animal& a) {
    a.speak();
    a.motion();
}
```

9.6 模板方法思想

模板方法模式在基类中固定算法骨架，把某些步骤延迟给子类：

```
class Base {
public:
    void action() {
        step1();
        step2();
        step3();
    }
protected:
    virtual void step1() {}
}
```

```
virtual void step2() {}  
virtual void step3() {}  
};
```

这是后续设计模式的重要铺垫。

9.7 函数模板

函数模板把类型也作为参数：

```
template <typename T>  
T sum(T a, T b) {  
    return a + b;  
}
```

调用时编译器实例化：

```
sum(1, 2);           // T=int  
sum(1.0, 2.0);       // T=double
```

若多个参数类型不一致，可能无法推导：

```
sum(1, 2.0); // 可能编译错误
```

模板参数类型必须支持模板内部使用的操作。例如排序模板要求元素支持 `>` 或给出比较方式。

9.8 类模板

```
template <typename T>  
class A {  
    T data;  
public:  
    A(T d) : data(d) {}  
};
```

成员函数类外定义：

```
template <typename T>  
void A<T>::print() {}
```

模板参数可为类型参数，也可为非类型参数：

```
template <typename T, unsigned size>
class Array {
    T elems[size];
};
```

非类型参数必须在编译期确定，不能用运行时变量。

9.9 模板原理与静多态

模板在编译期实例化：每出现一种模板参数组合，编译器生成一份对应代码。因此模板定义通常必须放在头文件中，不能像普通函数一样只把声明放头文件、定义放 `.cpp`。

模板多态是静态多态：编译期确定调用代码，效率高但可能导致代码膨胀。虚函数多态是动态多态：运行期绑定，灵活但有间接调用开销。

OOP 核心思想可概括为：

- 数据抽象：接口与实现分离。
- 继承：建立相关类型层次。
- 动态绑定：通过基类指针/引用实现多态行为。

第 10 讲 模板与 STL 初步

10.1 命名空间

命名空间用于控制标识符作用域，避免大型工程和库之间命名冲突：

```
namespace A {
    int x, y;
}

A::x = 3;
```

`using` 用法：

```
using namespace A; // 引入整个命名空间
using A::x;         // 只引入 x
```

标准库大部分内容位于 `std` 命名空间。大型工程不推荐在头文件中写 `using namespace std;`，会污染包含者命名空间。

10.2 STL 总体思想

STL 是标准模板库，主要包含：

- 容器：保存数据的数据结构。
- 迭代器：统一访问容器元素。
- 算法：在迭代器范围上操作。
- 函数对象：可被调用的对象，定制算法行为。

核心理念：把“数据结构”和“作用于数据的算法”解耦。

10.3 pair 与 tuple

pair 保存两个成员：

```
std::pair<std::string, double> p("Alice", 90.5);
p.first;
p.second;
auto q = std::make_pair("Bob", 85.0);
```

比较规则：先比 first，再比 second，要求成员类型支持比较。

tuple 是 C++11 的多元组：

```
auto t = std::make_tuple("abc", 7.8, 123);
auto x = std::get<0>(t);
```

std::get<index> 的下标必须编译期确定，不能使用运行时变量。

tie 可用于拆包：

```
std::string x;
double y;
int z;
std::tie(x, y, z) = std::make_tuple("abc", 7.8, 123);
```

10.4 vector

vector 是可自动扩容的连续数组：

```
std::vector<int> v;
v.push_back(1);
v.pop_back();
v.size();
v.clear();
v.insert(v.begin() + 1, 5);
v.erase(v.begin());
```

特点：

- 支持下标随机访问，速度快。
- 尾部插入均摊 $O(1)$ 。
- 中间插入/删除 $O(n)$ 。
- 扩容时通常申请更大内存并整体搬移，导致所有迭代器失效。

10.5 迭代器

迭代器类似指针，提供统一遍历接口：

```
for (std::vector<int>::iterator it = v.begin(); it != v.end(); ++it) {
    std::cout << *it << std::endl;
}

for (auto it = v.begin(); it != v.end(); ++it) {}
for (auto& x : v) {}
```

`begin()` 指向第一个元素，`end()` 指向最后一个元素之后的位置，构成左闭右开区间 `[begin, end)`。不能解引用 `end()`。

10.6 迭代器失效

迭代器失效指迭代器不再指向原本应指向的元素。`vector` 常见情况：

- `insert/erase` 后，修改位置及之后迭代器失效。
- `push_back` 等改变大小时若触发扩容，所有迭代器失效。

绝对安全准则：修改容器后，不再使用修改前保存的迭代器；必要时查文档的 `iterator validity`。

10.7 list

`list` 是双向链表：

```
std::list<int> l;
l.push_front(1);
l.push_back(2);
auto it = std::find(l.begin(), l.end(), 2);
l.insert(it, 3);
l.erase(it);
```

特点：

- 不支持下标随机访问。
- 任意位置插入/删除快。
- 插入删除通常不使其他迭代器失效，除被删除元素对应迭代器。

10.8 set

`set` 是不重复元素集合，内部按比较器维护顺序：

```
std::set<int> s;  
s.insert(3);  
auto it = s.find(3);  
s.count(3);  
s.erase(it);
```

课件称“无序集合”时强调不保持插入顺序；但 `std::set` 内部按大小有序，底层通常是红黑树，操作复杂度 $O(\log n)$ 。

10.9 map

`map` 是键值映射：

```
std::map<std::string, std::string> M;  
M["fp"] = "c";  
M["oop"] = M["fp"] + "++";
```

特点：

- key 唯一。
- `operator[]` 若 key 不存在，会创建默认值元素。
- 查找用 `find`，统计用 `count`。
- 底层通常为红黑树，操作复杂度 $O(\log n)$ 。

10.10 容器选择

- 需要高效随机访问、尾部追加：`vector`。
- 需要频繁中间插入删除且不随机访问：`list`。
- 需要按 key 查找、去重：`set`。
- 需要 key 到 value 映射：`map`。
- 关注迭代器稳定性时，`list` 和关联容器通常优于 `vector`。

10.11 string

`std::string` 是变长字符串类：

```
std::string s0("Initial string");  
std::string s1;  
std::string s2(s0, 8, 3);  
std::string s4(10, 'x');
```

常用操作：

```
s.size();  
s.length();
```

```
s.clear();
s.empty();
s.push_back('a');
s += "abc";
s[1] = 'x';
```

输入：

```
std::cin >> firstname;           // 读到空白
std::getline(std::cin, fullname); // 读一整行
std::getline(std::cin, part, ','); // 读到分隔符
```

数值转换：

```
std::to_string(3.14);
int a = std::stoi("2001");
double d = std::stod("3.14");
```

注意字符串反复拼接可能导致较高复杂度。

10.12 模板特化

函数模板可全特化：

```
template <typename T>
T sum(T a, T b);

template <>
char* sum<char*>(char* a, char* b);
```

函数模板不能部分特化，可用重载替代。

类模板可全特化和部分特化：

```
template <typename T1, typename T2>
class A {};

template <>
class A<int, int> {};

template <typename T1>
class A<T1, int> {};
```

函数模板匹配顺序要点：普通函数匹配优先于模板；模板全特化只有在对应基础模板被选中后才使用。因此函数模板全特化容易产生反直觉结果，实际中常用重载替代。

第 11 讲 iostream、函数对象、智能指针、正则

11.1 iostream 与文件流

常见流：

- `std::cin`：标准输入。
- `std::cout`：标准输出。
- `std::cerr`：标准错误输出。
- `std::ifstream`：文件输入。
- `std::ofstream`：文件输出。
- `std::fstream`：文件输入输出。

文件读写示例：

```
#include <fstream>

std::ifstream fin("input.txt");
std::ofstream fout("output.txt");

int x;
while (fin >> x) {
    fout << x << '\n';
}
```

复习重点：

- 流对象可用于条件判断，表示当前流状态是否正常。
- `>>` 读格式化数据，会跳过空白。
- `getline` 读整行。
- 输出流运算符重载返回 `ostream&`，支持链式输出。

11.2 函数指针

函数可通过指针传递：

```
int add(int a, int b) { return a + b; }
int (*fp)(int, int) = add;
fp(1, 2);
```

函数指针可作为策略传入算法，但类型写法复杂，无法统一函数对象、lambda 等不同可调用实体。

11.3 函数对象

函数对象是重载了 `operator()` 的对象：

```
class Greater {  
public:  
    bool operator()(int a, int b) const {  
        return a > b;  
    }  
};
```

对象可像函数一样调用：

```
Greater g;  
g(3, 2);
```

函数对象优势：

- 可保存状态。
- 可被模板参数接受。
- STL 算法广泛使用。

自定义排序：

```
std::sort(v.begin(), v.end(), Greater());
```

对自定义类型排序的三种方式：

1. 重载 `operator<`。
2. 定义比较函数。
3. 定义比较函数对象。

11.4 `std::function`

`std::function` 位于 `<functional>`，为函数指针、函数对象、lambda 提供统一包装：

```
#include <functional>  
  
std::function<int(int, int)> f = add;  
std::function<bool(int, int)> cmp = Greater();
```

意义：

- 函数对象化，符合“万物皆对象”的思想。
- 可把函数作为参数传递、作为变量保存。
- 避免为不同可调用类型写复杂模板。

11.5 模板、虚函数、`std::function` 对比

方式	绑定时机	优点	代价
虚函数	运行期	接口稳定，运行时灵活	需要类层次，有虚调用开销
模板	编译期	高效，支持 duck typing	代码膨胀，类型统一困难
<code>std::function</code>	运行期封装	统一函数、函数对象、lambda	有类型擦除和间接调用开销

11.6 智能指针和引用计数

智能指针来自 `<memory>`，封装动态内存管理，减少手动 `delete`。

`shared_ptr` 表示共享所有权：

```
std::shared_ptr<int> p1(new int(1));
auto p2 = std::make_shared<MyClass>(2);
auto p3 = p2;
std::cout << p2.use_count();
```

引用计数为 0 时自动销毁对象。

常用操作：

- `p.get()`：获取裸指针。
- `p.reset()`：清空并减少引用计数。
- `std::static_pointer_cast<T>(p)`。
- `std::dynamic_pointer_cast<T>(p)`。

11.7 `weak_ptr`

`shared_ptr` 可能产生循环引用：

```
Parent shared_ptr -> Child
Child shared_ptr -> Parent
```

即使外部指针销毁，两个对象互相持有引用计数，无法释放。解决：一方用 `weak_ptr`。

```
std::weak_ptr<Parent> parent;
```

`weak_ptr`：

- 指向对象但不增加引用计数。
- `expired()` 判断对象是否已无效。
- `lock()` 可尝试获得 `shared_ptr`。
- `reset()` 清除。

11.8 unique_ptr

unique_ptr 表示独占所有权：

```
std::unique_ptr<int> p(new int(3));  
auto q = std::make_unique<MyClass>(); // C++14
```

特点：

- 不可拷贝，只能移动。
- 无引用计数，性能较好。
- 语义明确：对象只由一个指针拥有。

选择：

- 默认优先 unique_ptr。
- 需要共享所有权时用 shared_ptr。
- 打破共享环或观察对象时用 weak_ptr。

11.9 正则表达式

<regex> 支持正则匹配、搜索、替换。

基本模式：

- [a-z]：小写字母。
- [0-9] / \d：数字。
- \w：字母、数字、下划线。
- +：一次或多次。
- *：零次或多次。
- {n,m}：重复 n 到 m 次。
- ^、\$：开头、结尾。
- ()：捕获分组。

C++ 字符串中反斜杠要转义：

```
std::regex re("\\d+");
```

可用原生字符串：

```
std::regex re(R"(\d+)");
```

匹配整个字符串：


```
std::regex_match(s, re);
```

搜索子串：

```
std::smatch result;  
std::regex_search(s, result, re);
```

替换：

```
std::regex_replace(s, re, "replacement");
```

捕获分组中 `result[0]` 是整体匹配，`result[1]` 起为各组。

第 12 讲 行为型设计模式

12.1 设计模式分类

设计模式关注可复用的软件设计经验。课程涉及：

- 创建型模式：对象创建过程。
- 结构型模式：对象组合结构。
- 行为型模式：对象间行为、职责和算法变化。

本讲行为型模式关注：抽象行为中不变部分，把可变步骤或策略延迟到子类或组合对象中。

12.2 模板方法模式

定义：在基类的一个方法中定义算法骨架，将某些步骤延迟到子类中，使子类可在不改变算法结构的情况下重新定义步骤。

结构：

```
class Monitor {
public:
    void run() {
        getLoad();
        getTotalMemory();
        display();
    }
protected:
    virtual void getLoad() = 0;
    virtual void getTotalMemory() = 0;
    virtual void display() = 0;
};
```

适用场景：

- 算法整体流程稳定。
- 某些步骤在不同平台或子类中变化。
- 希望公共流程只写一次。

体现原则：

- 针对接口编程。
- 开放封闭原则：新增实现类，不改旧流程。

缺点：若变化维度很多，所有步骤组合可能导致子类数量膨胀。

12.3 策略模式

定义：定义一系列算法并封装，使它们可互相替换。

结构：

```
class LoadStrategy {
public:
    virtual float getLoad() = 0;
    virtual ~LoadStrategy() {}
};

class MemoryStrategy {
public:
    virtual long getTotal() = 0;
    virtual ~MemoryStrategy() {}
};

class Monitor {
    LoadStrategy* loadStrategy;
    MemoryStrategy* memStrategy;
public:
    Monitor(LoadStrategy* l, MemoryStrategy* m)
```

```
        : loadStrategy(l), memStrategy(m) {}  
};
```

适用场景：

- 算法族独立变化。
- 需要在运行时组合不同算法。
- 变化维度之间可自由组合。

体现原则：

- 单一责任原则：每个策略类只负责一种算法。
- 组合优于继承。

模板方法 vs 策略：

比较	模板方法	策略
变化位置	子类覆盖步骤	组合不同策略对象
适合场景	流程稳定、部分步骤变化	算法族灵活组合
复用方式	继承	组合
类数量风险	多维变化时子类膨胀	策略类数量按维度增长

12.4 迭代器模式

定义：提供一种方法顺序访问聚合对象中各元素，而不暴露其内部表示。

目标：把“遍历算法”和“底层数据结构”解耦。例如上层分析成绩的算法不应依赖成绩存储在数组还是链表。

接口示例：

```
class Iterator {  
public:  
    virtual Iterator& operator++() = 0;  
    virtual bool operator!=(const Iterator& other) const = 0;  
    virtual float& operator*() = 0;  
    virtual ~Iterator() {}  
};
```

Collection 返回 [begin, end)：

```
class Collection {  
public:  
    virtual Iterator* begin() = 0;  
    virtual Iterator* end() = 0;  
};
```

算法只依赖迭代器：

```
void analyze(Iterator* begin, Iterator* end) {  
    for (Iterator* p = begin; *p != *end; ++(*p)) {  
        // use **p  
    }  
}
```

STL 迭代器也实现同一思想，但主要通过模板而非继承实现，是静态多态版本。

总结：

- 迭代器模式隔离算法和数据存储。
- 避免为每种算法和每种数据结构组合重复实现。
- STL 容器、算法、迭代器体系是该思想的重要实践。

第 13 讲 结构型设计模式

13.1 结构型模式概念

结构型模式关注类和对象的组合结构，包括接口转换、层次组织、对象包装等。目标是在结构层面降低耦合，提高复用和扩展能力。

本讲模式：

- 适配器 Adapter。
- 代理/委托 Proxy。
- 装饰器 Decorator。

13.2 适配器模式

定义：把一个类的接口转换成客户希望的另一个接口，使原本接口不兼容的类可以一起工作。

例子：已有 `std::vector<int>`，希望它像 `Stack` 一样提供 `push/pop/top/empty`。

目标接口：

```
class Stack {  
public:  
    virtual void push(int) = 0;  
    virtual void pop() = 0;  
    virtual int top() = 0;  
    virtual bool empty() = 0;  
    virtual ~Stack() {}  
};
```

对象适配器：用组合实现。

```
class Vector2Stack : public Stack {
    std::vector<int> data;
public:
    void push(int x) override { data.push_back(x); }
    void pop() override { data.pop_back(); }
    int top() override { return data.back(); }
    bool empty() override { return data.empty(); }
};
```

类适配器：用继承实现，常见做法是私有继承被适配类并公开目标接口：

```
class Vector2Stack : private std::vector<int>, public Stack {
    // 用 vector 的能力实现 Stack 接口
};
```

优点：

- 复用已有类。
- 客户端使用统一接口。
- 目标类和被适配器解耦，无需修改原类。

适配器只做接口转换，不负责额外行为控制。

13.3 代理/委托模式

定义：为某对象提供代理对象，由代理控制对原对象的访问。

适用场景：

- 远程代理：访问远程资源。
- 虚代理：对象创建开销大，延迟创建。
- 保护代理：访问控制。
- 智能引用：引用计数、自动释放等。

智能指针引用计数可看作代理思想：代理对象提供类似裸指针的接口 `*`、`->`，同时额外维护计数，在计数为 0 时释放资源。

代理和适配器区别：

比较	适配器	代理
目的	转换接口	控制访问
接口	通常变成目标接口	通常保持原对象接口
额外职责	主要是适配	可做权限、缓存、延迟、计数等

13.4 装饰器模式

定义：创建装饰类包装原有对象，在保持接口完整的前提下动态增加功能。装饰类和被装饰类继承同一基类，因此装饰后的对象可继续被装饰。

结构：

```
class Component {
public:
    virtual void draw() = 0;
    virtual ~Component() {}
};

class TextView : public Component {
public:
    void draw() override { /* draw text */ }
};

class Decorator : public Component {
protected:
    Component* component;
public:
    Decorator(Component* c) : component(c) {}
    void draw() override { component->draw(); }
};

class Border : public Decorator {
public:
    Border(Component* c) : Decorator(c) {}
    void draw() override {
        Decorator::draw();
        // draw border
    }
};
```

装饰器的关键是链式包装：

```
TextView view;
Border bordered(&view);
HScroll scrolled(&bordered);
```

每个装饰器只知道自己包裹的对象，不需要知道整条链。

13.5 装饰器、策略、代理对比

装饰器 vs 策略：

- 相同：都用组合改变行为。
- 策略通常替换某个算法维度，策略数量在基类结构中预先设计。
- 装饰器可不断叠加新功能，不需要提前在基类列出所有功能维度。

装饰器 vs 代理：

- 都包装对象并改变行为。
- 代理强调访问控制或管理。
- 装饰器强调在原接口基础上动态增加功能。

结构型模式总结：

- 适配器提升接口兼容和复用。
- 代理降低访问者与真实对象的耦合，增加控制逻辑。
- 装饰器动态扩展功能，避免继承组合爆炸。

期末高频易错点清单

1. `ClassName c();` 是函数声明，不是默认构造对象。
2. 初始化列表执行顺序按成员声明顺序，不按列表书写顺序。
3. `new[]` 必须配 `delete[]`，`new` 必须配 `delete`。
4. 隐式析构函数不会释放裸指针指向的堆内存。
5. 类含裸指针时默认拷贝构造和赋值通常危险。
6. 右值引用变量本身是左值。
7. `std::move` 只做类型转换，不实际搬资源；资源转移发生在移动构造/移动赋值中。
8. 基类 `private` 成员派生类也不能直接访问。
9. 派生类同名函数会隐藏基类所有同名重载，可用 `using Base::f` 恢复。
10. 多态必须通过基类指针或引用调用虚函数，对象赋值会切片。
11. 基类析构函数若用于多态删除，应设为 `virtual`。
12. 抽象类不能实例化，但可用指针或引用。
13. `dynamic_cast` 需要多态类型，转换失败指针返回 `nullptr`。
14. 模板定义通常放头文件，因为实例化发生在编译期。
15. `vector::end()` 指向最后一个元素之后，不能解引用。
16. `vector` 扩容会导致所有迭代器失效。
17. `map[key]` 在 `key` 不存在时会插入默认值。
18. `using namespace std;` 不应写在公共头文件中。
19. 友元不对称、不传递、不继承。
20. `static` 成员函数没有 `this`，不能直接访问非静态成员。
21. 常量对象只能调用 `const` 成员函数。
22. 流运算符重载应返回流引用以支持链式调用。
23. 函数模板不能部分特化，类模板可以部分特化。
24. 函数模板全特化不一定优先于普通重载函数。
25. `shared_ptr` 可能循环引用，需用 `weak_ptr` 打破环。

代码模板速查

类的基本写法

```
class A {  
private:  
    int data;
```

```
public:
    A(int x = 0) : data(x) {}
    int get() const { return data; }
    void set(int x) { data = x; }
};
```

资源管理类（拷贝/移动/析构）

```
class Buffer {
    int* p;
    int n;
public:
    Buffer(int n) : p(new int[n]{}), n(n) {}
    ~Buffer() { delete[] p; }

    Buffer(const Buffer& rhs) : p(new int[rhs.n]), n(rhs.n) {
        std::copy(rhs.p, rhs.p + n, p);
    }

    Buffer& operator=(const Buffer& rhs) {
        if (this != &rhs) {
            int* np = new int[rhs.n];
            std::copy(rhs.p, rhs.p + rhs.n, np);
            delete[] p;
            p = np;
            n = rhs.n;
        }
        return *this;
    }

    Buffer(Buffer&& rhs) : p(rhs.p), n(rhs.n) {
        rhs.p = nullptr;
        rhs.n = 0;
    }

    Buffer& operator=(Buffer&& rhs) {
        if (this != &rhs) {
            delete[] p;
            p = rhs.p;
            n = rhs.n;
            rhs.p = nullptr;
            rhs.n = 0;
        }
        return *this;
    }
};
```

多态接口


```
class Base {
public:
    virtual void f() = 0;
    virtual ~Base() {}
};

class Derived : public Base {
public:
    void f() override {}
};
```

函数模板和类模板

```
template <typename T>
T sum(T a, T b) { return a + b; }

template <typename T, unsigned N>
class Array {
    T data[N];
public:
    T& operator[](unsigned i) { return data[i]; }
};
```

STL 遍历

```
for (auto it = v.begin(); it != v.end(); ++it) {
    std::cout << *it << std::endl;
}

for (const auto& x : v) {
    std::cout << x << std::endl;
}
```

智能指针

```
auto p = std::make_shared<A>();
std::weak_ptr<A> wp = p;
if (auto sp = wp.lock()) {
    sp->f();
}

std::unique_ptr<A> up(new A());
auto up2 = std::move(up);
```

建议复习顺序

1. 先复习第 3 到第 6 讲：类、构造析构、引用、拷贝移动是后续所有内容的基础。
2. 再复习第 7 到第 9 讲：继承、虚函数、多态和模板是 OOP 主干。
3. 然后复习第 10 到第 11 讲：STL、函数对象、智能指针是工程实践重点。
4. 最后复习第 12 到第 13 讲：用模板方法、策略、迭代器、适配器、代理、装饰器串起“如何应对变化”的设计思想。
5. 第 1 到第 2 讲重点掌握编译链接、声明定义、头文件、Makefile、调试，它们常出现在代码阅读和工程题中。